MOBL-06: Crash Reporting & ANR Detection

> **Series:** MOBL — Mobile Monitoring | **Notebook:** 6 of 12 | **Created:** February 2026 | **Last Updated:** 10/14/2026

Overview

Mobile app crashes are the single most damaging event for user retention and app store ratings. Dynatrace automatically captures, symbolizes, and groups crashes across iOS and Android -- including Android Application Not Responding (ANR) events. This notebook explains how crash reporting works end-to-end, how to configure symbolication for readable stack traces, and how to query crash data with DQL to identify trends and prioritize fixes.

Table of Contents

1. How Crash Reporting Works
2. iOS Symbolication (dSYM)
3. Android Crash & ANR Detection
4. Crash Grouping
5. Querying Crash Data
6. Crash Rate Trends
7. Manual Error Reporting

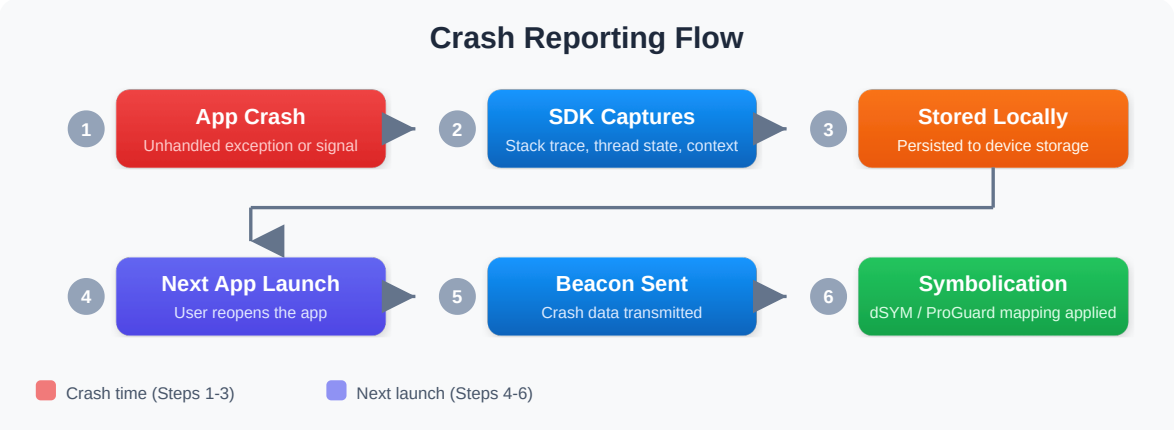
Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`storage:user.events:read`, `storage:user.sessions:read`

Requirement	Details
Mobile App	At least one mobile application with crash reporting enabled
Symbolication	dSYM files (iOS) or ProGuard/R8 mapping files (Android) uploaded
Prior Knowledge	Familiarity with MOBL-01 through MOBL-05 recommended

1. How Crash Reporting Works

When a mobile app crashes, the Dynatrace SDK captures the crash details and sends them to the backend for processing. The process differs slightly between handled exceptions and unhandled crashes, but the overall flow is the same.



Crash data is captured at crash time and sent on the next app launch.

Handled vs Unhandled Crashes

It is important to understand the distinction between handled exceptions and unhandled crashes:

Type	Description	SDK Behavior
Unhandled Crash	The app terminates unexpectedly due to an unhandled exception (e.g., `NullPointerException`, `EXC_BAD_ACCESS`) or fatal signal	SDK crash handler fires and captures the stack trace. Usually sent immediately; otherwise on a relaunch within 10 minutes.
Handled Exception	The developer catches an error in a try/catch block and explicitly reports it via the SDK API	SDK sends the error beacon immediately (app is still running). Does not terminate the app.

Key difference: Unhandled crashes terminate the app; their report is usually sent immediately, with a relaunch within 10 minutes as the fallback. Handled exceptions are reported while the app continues running.

> **Note (Android-documented):** When the immediate send fails and the user does not reopen the app within 10 minutes, the crash report is never sent, because Dynatrace does not send crash reports older than 10 minutes. Crash counts can therefore under-represent severe crashes that make users abandon the app. The 10-minute window is documented for the Android Gradle plugin; confirm the iOS behavior on the OneAgent for iOS pages before relying on it there.
>
> **Correction (09/28/2026).** Earlier revisions said crash beacons are always stored and sent "on the next launch". The documentation says they are generally sent immediately.
>
> **Sources:** Configure monitoring capabilities (DT docs) — *"In general, the crash details are sent immediately after the crash, so the user doesn't have to relaunch the application. However, in some cases, the application should be reopened within 10 minutes so that the crash report is sent."*

2. iOS Symbolication (dSYM)

When an iOS app is compiled in Release mode, the compiler strips debug symbols from the binary to reduce its size. These symbols are stored in **dSYM (Debug Symbol)** files. Without dSYM files, crash stack traces show only memory addresses like `0x00000001045a3b2c` instead of meaningful method names like `-[PaymentViewController processPayment:]`.

What Are dSYM Files?

A dSYM file is a companion file generated during the Xcode build process that maps memory addresses back to source code symbols (class names, method names, file names, and line numbers). Each build produces a unique dSYM file identified by a UUID that must match the app binary.

Upload Methods

iOS dSYM files are **preprocessed with the DSSClient** (shipped with OneAgent for iOS) into symbol extract files before Dynatrace accepts them. Four upload routes are documented:

Method	Description	Best For
DSSClient	`DTXDssClient -decode` then `DTXDssClient -upload` — scriptable in an Xcode run-script phase or CI	CI/CD pipelines
REST API	Mobile Symbolication API, `PUT /api/config/v1/symfiles/...` with a `DssFileManagement` token (upload the DSSClient's zip)	Custom CI/CD systems

Method	Description	Best For
Fastlane	The Dynatrace Fastlane plugin automates processing and upload	Teams already using Fastlane
Web UI	**Settings > Web and mobile monitoring > Source maps and symbol files → iOS → Upload files**	Ad-hoc builds, troubleshooting

DSSClient Example

1. Preprocess the dSYMs (from the `.xcarchive`, or a dSYM zip from App Store Connect):

```
DTXDssClient -decode symbolsfile=MyApp.xcarchive
```

2. Upload the symbol extract files:

```
DTXDssClient -upload appid=&lt;APPLICATION_ID&gt; apitoken=&lt;API_TOKEN&gt; os=ios \
  bundleId=com.yourcompany.app bundleName=MyApp versionStr=1.0 version=1 \
  symbolsfile=MyApp.xcarchive/dSYMs server=https://&lt;your-environment-
id&gt;.live.dynatrace.com
```

`versionStr` and `version` must match the app's `CFBundleShortVersionString` and `CFBundleVersion`. Only stack-trace lines from your app and third-party libraries you supply dSYMs for are symbolicated — **system library frames are not**.

> **Correction (10/02/2026).** Earlier revisions showed `DTXDssClient -ApiToken … -DtApplicationID … -Server …` flags. The documented syntax is `key=value` pairs after `-upload`, as above.
>
> **Sources:** Upload and manage symbol files (DT docs) — *"For iOS or tvOS symbolication, you need to preprocess dSYM files using the DSSClient before you can upload them to Dynatrace."*; *"Symbolication of system library stack trace lines is not supported."*

Bitcode Considerations

If your app uses Bitcode (now deprecated in Xcode 14+), Apple recompiles the binary on their servers before distributing to users. This means the dSYM you generate locally does **not** match the binary users actually run. You must download the App Store-generated dSYMs from App Store Connect (or via Xcode Organizer) and upload those to Dynatrace instead.

Symbolication Status

Stack Trace Appearance	Meaning
`MyApp -[PaymentViewController processPayment:] + 42`	Fully symbolicated -- dSYM uploaded correctly
`MyApp 0x00000001045a3b2c`	Not symbolicated -- dSYM missing or UUID mismatch
`MyFramework 0x00000001089d1f30`	Framework dSYM missing -- upload framework symbols too
`libsystem_kernel.dylib 0x...`	System library frame -- not symbolicated by Dynatrace, by design

> **Tip:** Always verify symbolication status after uploading dSYMs. If crashes still show memory addresses, check that the dSYM UUID matches the app binary UUID using `dwarfdump --uuid` on both files.

3. Android Crash & ANR Detection

Android crash reporting covers three distinct types of failures, each with different detection mechanisms:

Type	Description	Detection
Unhandled Exception	`RuntimeException`, `NullPointerException`, `OutOfMemoryError`, etc.	Automatic -- SDK installs an `UncaughtExceptionHandler`
ANR (Application Not Responding)	The system's ANR condition — for input events, no response within about 5 seconds	Reported by OneAgent for Android (`characteristics.has_anr` in Grail)
Native Crash	Segfault or other fatal signal in NDK/C++ code (e.g., `SIGSEGV`, `SIGABRT`)	Reported with its signal in `exception.crash_signal_name`; NDK code itself is not instrumented. Check your agent version's crash-reporting settings

ANR Detection in Detail

An ANR occurs when the Android system detects that the main (UI) thread has been unresponsive for more than 5 seconds. Common causes include:

- **Database operations on the main thread** -- SQLite queries, Room transactions
- **Network calls on the main thread** -- Synchronous HTTP requests
- **Heavy computation** -- Image processing, JSON parsing of large payloads
- **Lock contention** -- Main thread waiting for a lock held by a background thread
- **Deadlocks** -- Two threads waiting on each other

The ANR record carries the stack trace where the main thread was blocked, alongside the session and user actions that led up to it — context the platform's own ANR counts do not give you.

ProGuard/R8 Mapping Files

When you enable code shrinking with ProGuard or R8 (the default for release builds), class and method names are obfuscated. Without the mapping file, crash stack traces show names like `a.b.c.d()` instead of `com.example.PaymentService.processPayment()`.

Upload mapping files through any of the documented routes — none of them is automatic in the Gradle build:
- **DSSClient** — `DTXDssClient -upload appid=… apitoken=… os=android bundleId=… versionStr=… version=… file=…/mapping.txt server=…`
- **REST API** — Mobile Symbolication API `PUT /api/config/v1/symfiles/{applicationId}/{packageName}/ANDROID/{versionCode}/{versionName}` with a `DssFileManagement` token (MOBL-03 §6 has a `curl` example)
- **Fastlane** — the Dynatrace Fastlane plugin
- **Web UI** — **Settings > Web and mobile monitoring > Source maps and symbol files** → **Android** → **Upload files** (package name, version code, version name, mapping file)

Source maps and symbol files share a 1 GiB storage quota per SaaS environment; when it fills, older files are deleted automatically unless **pinned**.

> **Correction (10/02/2026).** Earlier revisions showed a Gradle `symbols { apitoken '…' }` block that uploads mapping files during the build. That block is not documented; the upload routes are the four above.
>
> **Sources:** Upload and manage symbol files (DT docs) — *"The Dynatrace platform supports four different ways of uploading Android mapping files and iOS or tvOS symbol extract files"*; *"For Dynatrace SaaS, the maximum storage size for source maps and symbol files is 1 GiB."*; Mobile Symbolication API — PUT (DT docs) — *"you need an access token with DssFileManagement scope"*.

> **Important:** Upload mapping files for every release build. If a mapping file is missing for a specific app version, crashes from that version will show obfuscated stack traces.

4. Crash Grouping

When Dynatrace receives crash reports, it does not treat each individual crash as a separate issue. Instead, it **groups crashes by root cause** to help you focus on the most impactful problems.

How Grouping Works

Each crash record carries `error.id`, a fingerprint that groups similar crashes; the exact fingerprint inputs are not published, but in community practice crashes with the same exception type and the same crashing frame land in the same group.

This means that 500 users crashing on the same `NullPointerException` in `PaymentViewController.processPayment()` will appear as a **single crash group** with a count of 500, rather than 500 individual crash entries.

Benefits of Crash Grouping

Benefit	Description
Prioritization	See which crashes affect the most users and fix those first
Trend tracking	Monitor whether a specific crash is increasing or decreasing across versions
Regression detection	Quickly identify if a new app version introduced a new crash group
Noise reduction	Hundreds of individual crash reports collapse into actionable groups

Crash Fields in Grail

Crashes are `user.events` records with `characteristics.has_crash` (ANRs: `characteristics.has_anr`). Both are set only by OneAgent for Mobile. The fields that identify and describe a crash (semantic dictionary model `rum_crash`, read 09/28/2026):

Field	Description
`error.id`	Fingerprint that groups similar crashes -- the crash group
`exception.type` / `exception.message`	Exception class and message
`exception.stack_trace`	Full stack trace

Field	Description
`exception.crash_signal_name`	Signal for native crashes (for example `SIGSEGV`)
`error.is_fatal`	Whether the error terminated the app
`frontend.name`	The mobile application
`app.short_version` / `app.version`	App version (store version / full build)
`os.name`	`iOS` or `Android`

At session grain, `user.sessions` carries `error.has_crash` and `error.anr_count`, which is what a crash **rate** needs (crashed sessions / sessions).

> **Tip:** Use `summarize crashes = count(), by:{error.id, exception.type}` to see crash groups ranked by frequency. This immediately shows which crash should be fixed first.

5. Querying Crash Data

Crash data is stored in the Grail `user.events` store (`characteristics.has_crash`), not in business events. Use the following queries to explore recent crashes, identify the most affected applications, and visualize crash trends over time.

Recent Mobile Crashes

Retrieve the 50 most recent crash events across all mobile applications:

```
// Recent mobile crashes (characteristics.has_crash is set only by OneAgent for
Mobile)
fetch user.events, from:-24h
| filter characteristics.has_crash
| fields start_time, frontend.name, os.name, app.short_version, exception.type,
exception.message
| sort start_time desc
| limit 50
```

Crash Count by Application

See which applications and platforms have the highest crash volume over the last 7 days:

```
// Crash count by application over last 7 days
fetch user.events, from:-7d
| filter characteristics.has_crash
| summarize {crash_count = count(), affected_sessions =
countDistinct(dt.rum.session.id)}, by:{frontend.name, os.name}
| sort crash_count desc
```

Daily Crash Trend

Visualize how overall crash volume is trending day-over-day. Spikes may indicate a problematic release:

```
// Daily crash trend
fetch user.events, from:-7d
| filter characteristics.has_crash
| makeTimeseries crash_count = count(), interval:24h
```

6. Crash Rate Trends

Looking at raw crash counts is useful, but breaking them down by platform reveals whether iOS or Android is experiencing more instability. The following queries help you compare crash rates across platforms and identify handled exceptions reported through the SDK API.

Crash Rate by Platform

Compare daily crash volumes between iOS and Android to spot platform-specific regressions:

```
// Crash timeseries by platform
fetch user.events, from:-7d
| filter characteristics.has_crash
| makeTimeseries crash_count = count(), by:{os.name}, interval:24h
```

Reported Errors (Handled Exceptions)

In addition to unhandled crashes, developers can report caught exceptions using the SDK API. These are `user.events` records with `characteristics.has_error` and `characteristics.is_api_reported` (model `rum_api_reported_error`), and represent errors that the app handled gracefully but that you still want visibility into:

```
// Reported errors (errors reported through the OneAgent SDK / Dynatrace API)
fetch user.events, from:-24h
| filter dt.rum.application.type == "mobile"
| filter characteristics.has_error and characteristics.is_api_reported
| fields start_time, frontend.name, error.name, error.code, error.reason, os.name
| sort start_time desc
| limit 50
```

7. Manual Error Reporting

While Dynatrace automatically captures unhandled crashes, you should also report **handled exceptions** that are important to your business logic. For example, a payment failure caught in a try/catch block does not crash the app, but you absolutely want visibility into it.

iOS (Swift)

```
// iOS - Report a handled error
let error = NSError(
    domain: "PaymentError",
    code: 402,
    userInfo: [NSLocalizedDescriptionKey: "Card declined"]
)
DTXAction.reportError(withName: "Payment Failed", error: error)
```

Android (Kotlin)

```
// Android - Report a handled error
Dynatrace.reportError("Payment Failed", Exception("Card declined"))
```

Best Practices for Manual Error Reporting

Practice	Rationale
Use descriptive error names	`"Payment Failed"` is better than `"Error"` -- it makes DQL grouping meaningful
Include context in the exception	Pass the underlying exception object so the stack trace is captured
Report at catch boundaries	Report where you handle the error, not deep in utility methods
Avoid over-reporting	Don't report expected conditions (e.g., network timeout during retry) as errors

Practice	Rationale
Use consistent naming	Establish naming conventions across your team so errors group predictably

> **Note:** Reported errors appear in `user.events` with `characteristics.has_error` and `characteristics.is_api_reported`, while unhandled crashes carry `characteristics.has_crash`. Use the appropriate filter depending on what you are investigating.

Summary

In this notebook, you learned:

- **How crash reporting works** -- from the moment the app crashes through SDK capture, transmission (usually immediate, otherwise on a relaunch within 10 minutes), server-side symbolication, and display in the UI
- **iOS symbolication** -- why dSYM files are essential, how to preprocess and upload them (DSSClient, REST API, Fastlane, web UI), and Bitcode considerations
- **Android crash types** -- unhandled exceptions, ANRs (main thread blocked >5 seconds), and native NDK crashes, plus ProGuard/R8 mapping file upload
- **Crash grouping** -- the `error.id` fingerprint that groups similar crashes so you can prioritize fixes by frequency
- **Querying crash data with DQL** -- reading `user.events` with `characteristics.has_crash`: fetching recent crashes, counting by application, and visualizing daily crash trends
- **Crash rate trends** -- comparing crash volumes across iOS and Android platforms, plus querying handled exceptions
- **Manual error reporting** -- using the SDK API to report handled exceptions in Swift and Kotlin with best practices for naming and context

---

Next Steps

Continue to **MOBL-07** to explore:
- Network request monitoring for mobile applications
- Analyzing HTTP error rates and response times by endpoint
- Correlating network failures with user impact and crash data

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.